

The RBLapSum surrogate in a code-residual stack

Research journal

Started 2026-09-30 (box lisbon), following
docs/stream-bottleneck-depth-journal.tex.

Contents

1	The question	1
2	Entry 1 — the backward grows toward the input	2
3	Entry 2 — where the growth comes from	2
4	Entry 3 — keeping the carry’s backward exact	3
5	Entry 4 — training screens	3
6	Entry 5 — the code’s scale sets the surrogate’s strength	4
7	Entry 6 — what fills the support: always-on features	6
8	Entry 7 — one support term per gradient path	8
9	Entry 8 — does the first-order estimator carry over?	10
10	Summary	11

1 The question

The code residual (`code_residual=true`) made a 24-layer stream-bottleneck model train with plain hard TopK (val CE 1.266 at 20k steps, $K = 512$). The next step was to repeat the K/J comparison of `docs/kj-vs-hard-topk.tex` – RBLapSum TopK with J extra backward-only candidates against hard Top- K' – in this architecture: 24 layers, $d = 1024$, $N = 4096$, MD, `bottleneck_init=standard`, `bottleneck_decoder_scale=none`, `through_rank_kappa`, $b_0 = 0$, shared projections, no post-norm, no value shift, $T \in \{1, 2\}$ (base config `/workspace/configs_lisbon/base_24L`).

A 60-step smoke run ($K = 32$, $J = 480$, $T = 1$) trained at 1.10s/step but with a gradient norm of 12 000–15 000 at every step, against ~ 1.5 for the hard code residual, and a much slower loss (10.96 at step 60 against 7.28). The campaign was held back to find the cause.

Notation. At gate ℓ the input is $u = c_\ell + \alpha W_E \Delta_\ell$ (carried code plus encoded update), the scores are $s_i = |u_i|$, the rank boundary is $b = s_{(K+1)}$, the pool is the Top- $(K + J)$, m_i the hard mask.

RBLapSum’s backward is the hard path plus a support term,

$$\frac{\partial \mathcal{L}}{\partial u_i} = m_i g_i + \text{sign}(u_i)(a_i - q_i \sum_j a_j), \quad a_i = g_i u_i \kappa_i, \quad \kappa_i = \frac{e^{-|s_i - b|/T}}{2T}, \quad q_i = \frac{\kappa_i}{\sum_j \kappa_j}, \quad (1)$$

with $g = \partial \mathcal{L} / \partial c_{\ell+1}$ the gradient arriving at the gate’s output.

2 Entry 1 — the backward grows toward the input

`analysis/depth_probe.py` at initialization, per-block factor of the fc1 weight gradient $(g_{23}/g_0)^{1/23}$ (below 1: the gradient grows toward the input):

cell	$T = 1$	$T = 2$
hard $K = 32$ / hard $K = 512$	1.020 / 1.011	
RBLapSum $K = 32, J = 32$	0.620	
RBLapSum $K = 32, J = 480$	0.619	0.677
RBLapSum $K = 256, J = 256$	0.662	0.712
$K = 32, J = 224: T = 4, 8, 16$	0.734, 0.789, 0.849	
$K = 32, J = 224, T = 1$, rblapsum_support_scale 0.3 / 0.1	0.908 / 0.992	
$K = 256, J = 256, T = 1$, rblapsum_support_scale 0.3 / 0.1	0.926 / 0.991	
$K = 32, J = 224, T = 1$, detach / project	0.617 / 0.642	
$K = 32, J = 224, T = 1$, lapsum (mass-constrained)	0.474	
$K = 32, J = 224, T = 1$, update scale $\alpha = 0.3$	0.777	

Measured: with the surrogate the gradient grows by a factor 1.4–1.6 per block toward the input (first block 5.7 against 10^{-4} for the last at $K = 32, J = 224, T = 1$). It depends little on J and on the boundary mode, falls only slowly with T (still 1.18 per block at $T = 16$), and disappears when the whole support term is scaled by 0.1. In the post-norm architecture the same surrogate was near-critical ($\times 1.07$ per block at $T = 1$, depth journal Entry 3).

3 Entry 2 — where the growth comes from

`analysis/surrogate_probe.py` splits the gradient at every gate into its hard part $m \odot g$ and the surrogate part (the rest), and records the boundary and the code scale. At $K = 32, J = 224, T = 1$:

gate	0 (entry)	1	6	12	18	24
boundary b	2.67	3.63	4.15	4.25	4.33	4.39
RMS of the kept scores	3.00	4.19	6.33	7.71	8.76	9.65
pool members with $ s - b < T$	255	189	84	63	53	49
$\sigma = \text{surrogate} / \text{hard energy}$	3.77	3.47	1.79	1.34	1.20	3.11
RMS of $\partial \mathcal{L} / \partial c$ (hard model)	$2.3 \cdot 10^{-6}$	$1.9 \cdot 10^{-6}$	$1.6 \cdot 10^{-6}$	$1.5 \cdot 10^{-6}$	$1.5 \cdot 10^{-6}$	$9.1 \cdot 10^{-6}$
RMS of $\partial \mathcal{L} / \partial c$ (RBLapSum)	0.60	0.24	$1.1 \cdot 10^{-2}$	$5.5 \cdot 10^{-4}$	$3.1 \cdot 10^{-5}$	$9.1 \cdot 10^{-6}$

Three facts follow. (i) The gradient on the carried code is dense: the next block reads the code through the decoder, $x = g_D W_D c$, so $\partial \mathcal{L} / \partial c = g_D W_D^\top \partial \mathcal{L} / \partial x$ reaches every coordinate, active or not (in the hard model 90% of its energy is off the support at gate 12). The hard gate passes only the K active coordinates; the surrogate also passes the pool. (ii) The kept scores are large compared with T : the boundary sits at 3.6–4.4 and the kept values at 4–10, because the code accumulates additively like a residual stream. (iii) The surrogate’s energy is comparable to or larger than the hard part at every gate ($\sigma = 1.2$ –3.8).

The mechanism. Per coordinate, the backward of one gate multiplies g_i by

$$\text{hard: } m_i \in \{0, 1\}, \quad \text{RBLapSum: } m_i + |u_i| \kappa_i \text{ (minus the rank-one correction),} \quad (2)$$

and the carry $u = c_\ell + \dots$ is the identity on the same coordinate. An active coordinate near the boundary is therefore multiplied by $1 + |u_i| \kappa_i \approx 1 + b/(2T) \approx 3$ at $T = 1$, at every gate it passes; since the code carries features from gate to gate, the same coordinates stay near the boundary over several gates and the factors compound. In the post-norm architecture every layer re-encodes into fresh coordinates, so no per-coordinate product forms (and the selection gain’s $1/s$ dominated). The table’s trend in T fits this: larger T lowers κ at the boundary but spreads it over more of the pool, and $1 + |u| \kappa > 1$ holds for every active coordinate at any T .

A smoothed forward is not the way out (`rblapsum_sf`, the soft forward whose backward is its exact gradient): $\times 0.717$ per block at $K = 32, J = 224, T = 1, \sigma = 4.8$. The soft gate $u \mapsto up((|u| - b)/T)$ has slope $p + |u| \kappa$, which exceeds 1 in a band around the boundary (about $1/2 + b/(2T) \approx 2.6$ at it): the smoothed stack genuinely amplifies perturbations along the carried coordinates. The growth is not an artefact of the mismatch between a hard forward and a soft backward; it comes from any smooth selection with slope above 1, compounded by an identity carry.

4 Entry 3 — keeping the carry’s backward exact

What must be preserved is the exact Jacobian of the carry (the mask, isometric on the support), while the surrogate keeps doing its job: giving gradient to the J candidates so the model learns what to select. Two ways, both prototyped as patches (`surr_fix_proto.py`, `train_patched.py` on the box):

- **(F1) local surrogate.** The support term is routed only into the block’s own update $\alpha W_E \Delta_\ell$; the carried code receives the exact hard gradient $m \odot g$. Each selection decision is credited once, to the block that makes it, rather than once per gate the feature passes. Implemented as $y = \text{gate}(\bar{c} + v) + (y_h - \bar{y}_h)$ with $\bar{\cdot}$ a detach and $y_h = (c + \bar{v}) \odot m$: the forward is unchanged, the carry gets $m \odot g$, the update gets the full surrogate gradient.
- **(F3) inactive-only surrogate.** The support term is kept for the inactive pool members and dropped on active ones, which then keep the exact multiplier 1 (the κ -weighted correction is taken over the inactive members).

Per-block factor of $\partial \mathcal{L} / \partial c$ at initialization (hard: $\times 1.07$, unmodified: $\times 0.64$):

	$K = 32, J = 224, T = 1$	$K = 32, J = 224, T = 2$	$K = 256, J = 256, T = 1$
unmodified	0.643 ($\sigma = 1.72$)	0.704 (0.66)	0.683 (0.87)
(F1) local	0.999 ($\sigma = 1.26$)	1.020 (0.57)	1.000 (0.51)
(F3) inactive	1.053 ($\sigma = 0.37$)	1.064 (0.17)	1.043 (0.09)

Measured: both remove the growth. F1 keeps the surrogate’s full energy at the gate (it is only routed away from the carry); F3 keeps a third to a tenth of it. **Interpretation:** the diagnosis holds — the growth is the surrogate term compounding along the carried coordinates.

5 Entry 4 — training screens

2000-step screens at $K = 32, J = 224$ unless marked, 24 layers, the campaign’s base config; the fixes were applied with the prototype entry `train_patched.py` (identical to `rblapsum_surrogate_scope`, landed later in `4beaf1b`). Hard code residual at $K = 32$ (`li_24L_k32_coderes_a1`) for reference.

run	val CE at step			dead features	usage entropy	boundary b
	500	1000	1500			
hard $K = 32$ (reference)	2.912	2.453	2.241	0.001	0.41	—
unmodified, $T = 1$	5.784	5.496	5.340	0.94	0.035	105–335
rblapsum_support_scale 0.1	2.890	2.428	2.216	0.000	0.64	3.9–2.4
(F3) inactive only, $T = 1$	2.912	2.467	2.256	0.006	0.45	5.3–4.3
(F1) local, $T = 1$	4.908	5.443	5.629	0.93	0.031	93–799
(F1) local, $T = 2$	3.467	5.208	5.344			
(F1) local, $J = 32$, $T = 1$	4.072	5.395	5.194			

Measured. The unmodified surrogate and F1 both collapse onto $\sim 6\%$ of the features (usage entropy 0.03), whose values run away (the rank boundary grows from 4 to hundreds), and the loss rises. F1 does so although its backward was flat at initialization. F3 is stable and equals hard within 0.015. A global support scale of 0.1 is stable, spreads the usage (entropy 0.64 against 0.41 for hard) and is ahead of hard by 0.02–0.025 at every evaluation.

What F1 taught. Removing the compounding from the backward is not enough: the support term $a_i = g_i u_i \kappa_i$ is proportional to the value u_i . For an *active* feature it makes the gradient on its value proportional to the value itself — a growth-rate term, $\dot{u} \propto u$ — and in a code that accumulates through an identity carry, that is exponential growth of whichever features the loss favours, until they win every selection. Routed into the update (F1) or into the carry (unmodified), the term runs away the same way. The post-norm architecture never showed this because every layer reset the scale. With an absolute T there is a second feedback: as the code grows, κ at the boundary stays $1/(2T)$ while $u \approx b$ grows, so the push itself grows with the code scale. F3 removes the active-feature term altogether and is stable, but gains nothing; a small dose of it on every member (scale 0.1) is stable and helps a little.

6 Entry 5 — the code’s scale sets the surrogate’s strength

Measured (the logs of Entry 4; b is the rank boundary averaged over the 25 gates and the batch, the usage entropy is normalized to 1 for uniform use):

run	b / usage entropy at step							
	1	100	200	300	400	500	1000	2000
(F1) local, $T = 1$	4.16/0.80	5.48/0.44	7.14/0.17	13.2/0.09	40.7/0.06	93.4/0.04	388/0.03	1179/0.03
(F1) local, $T = 2$	4.16/0.80	5.66/0.46	6.56/0.30	8.85/0.15	14.2/0.14	44.2/0.11	331/0.04	1911/0.03
unmodified, $T = 1$	4.16/0.80	7.29/0.39	14.1/0.06	37.1/0.04	64.3/0.04	105/0.03	189/0.04	470/0.04
support scale 0.1	4.16/0.80	5.15/0.48	4.53/0.62	4.32/0.67	4.15/0.69	3.86/0.68	2.76/0.65	2.23/0.62
(F3) inactive only	4.16/0.80	5.71/0.42	5.09/0.51	5.00/0.55	5.16/0.56	5.26/0.54	4.65/0.49	4.27/0.42

All runs lose usage entropy in the first 100 steps and their boundary rises to 5–6. In the stable runs the boundary then falls and the entropy partly recovers; in F1 and the unmodified run the boundary keeps growing, accelerating (F1 at $T = 1$: $\times 1.3$, $\times 1.9$, $\times 3.1$, $\times 2.3$ per 100 steps from step 100), while the usage concentrates and features die. F1’s training CE follows the stable runs to step ~ 150 (6.48 at 100 against 6.42–6.41) and departs as the boundary grows (4.13 at 200 against 4.07–4.08; 3.70 at 300 against 3.44–3.45; rising from 400).

Interpretation (a hypothesis to test). Hard TopK is 1-homogeneous, $\text{TopK}(\lambda u) = \lambda \text{TopK}(u)$. Every block reads the code through its pre-LN RMSNorm and the final norm removes the code’s scale, so the model’s function depends on the code scale only through the ratio of each update to the carry; the scale is weakly constrained. The surrogate with a fixed T is not scale-free: at the bound-

ary $|u|\kappa = b/(2T)$, and the exact surrogate gain of `docs/scale-dynamics-note-neutral.tex`,

$$\Pi = \|L_\kappa D_z\|_F^2 \approx \frac{b^2}{4T\delta}, \quad L_\kappa = D_\kappa - \frac{\kappa\kappa^\top}{\sum_i \kappa_i}, \quad (3)$$

(δ the local rank spacing at the boundary) grows linearly with the scale when the shape of the score distribution is fixed (b and δ both scale). That note found that a large Π drives large support updates that raise the score scale at second order in the step (“heating”). With T fixed, a larger scale is a larger Π , and the loop closes. In the post-norm stack each gate’s input is normalized; the code residual has no such normalization.

The fix to test: a scale-free kernel. `rb lapsum_relative_temperature` (commit 2914577) sets the kernel width to $T_{\text{eff}} = \tau b$ per row. Then $|u|\kappa = 1/(2\tau)$ at the boundary at every scale, $\Pi \approx b/(4\tau\delta)$ depends on the shape of the scores only, and scaling the gate input is an exact reparameterization of the whole backward (unit test). At initialization $b \approx 4.2$, so $\tau = 0.25$ is the $T = 1$ kernel: **prediction** — if the loop drives the collapse, F1 at $\tau = 0.25$ does not collapse, although it starts from exactly the state of the F1 run that did. `tools/repro_check.py`: IDENTICAL against `4beaf1b` (hard and rbk), so the default path is unchanged.

Round 2 (2000 steps, $K = 32$, $J = 224$, same base config):

run	change	tests
<code>upd_rel025</code>	F1, $\tau = 0.25$	the loop (same kernel as F1 $T = 1$ at step 0)
<code>upd_rel05</code> , <code>upd_rel1</code>	F1, $\tau = 0.5$, 1	strength at fixed shape
<code>upd_t1_ss03</code>	F1, $T = 1$, support scale 0.3	strength alone, without scale-freedom
<code>pool_t1_ss03</code> , <code>pool_t1_ss003</code>	unmodified, support scale 0.3, 0.03	the existing knob (0.1 was best so far)
<code>pool_t1_ss0</code>	unmodified, support scale 0	control: hard backward, boundary logged

Measured (val CE at steps 500/1000/1500/2000; usage entropy and dead fraction at step 600; σ : surrogate/hard energy at the gate, mean over gates, `surrogate_probe.py` at initialization):

run	σ at step 0	val CE 500 / 1000 / 1500 / 2000	entropy / dead (600)	b (600)
support scale 0 (exact hard backward)	0	2.896 / 2.435 / 2.232 / 2.104	0.61 / 0.000	4.2
F1, $\tau = 0.25$	1.37	2.948 / 2.675 / – / –	0.12 / 0.48	14.9
F1, $\tau = 0.5$	0.61	2.922 / 2.549 / – / –	0.15 / 0.40	11.9
F1, $\tau = 1$	0.26	2.899 / 2.491 / 2.350 / 2.258	0.18 / 0.30	10.7
F1, $T = 1$, support scale 0.3	0.087	2.871 / 2.424 / 2.250 / 2.189	0.26 / 0.008	5.5
unmodified, support scale 0.3	0.11	2.937 / 2.440 / 2.206 / 2.064	0.37 / 0.001	4.6
unmodified, support scale 0.1 (Entry 4)	0.010	2.890 / 2.428 / 2.216 / 2.089	0.67 / 0.000	4.3
unmodified, support scale 0.03	0.001	2.893 / 2.436 / 2.226 / 2.094	0.63 / 0.000	3.7
F3, $T = 1$ (Entry 4)	0.37	2.912 / 2.467 / 2.256 / 2.119	0.53 / 0.002	5.2

The prediction failed. F1 with the scale-free kernel collapses at every width: $\tau = 0.25$, which starts from the same kernel as the F1 $T = 1$ run, loses features at nearly the same pace (dead 0.40 at step 400 against 0.76), and $\tau = 1$, with a quarter of the gain at the boundary ($|u|\kappa = 0.5$) and $\sigma = 0.26$, still loses 30 % of the features by step 600. The growing scale is not what drives the collapse; with the scale removed from the kernel the boundary still grows (4.2 to 11–15). The width matters little and the total strength a lot: F1 at $T = 1$ with the whole support term scaled by 0.3 ($\sigma = 0.087$) is the best run at step 1000, though its usage entropy also falls. The F1 $\tau = 0.25$ and 0.5 runs were stopped at step 1140 (collapsing). F1 at support scale 0.3 leads at step 1000 and then falls behind (2.189 at 2000, +0.085), its usage still concentrating. Without the update routing, the unmodified surrogate at support scale 0.3 is the best run at step 2000, 0.040 below the exact hard backward, ahead of 0.1 (–0.014) and 0.03 (–0.009); its usage also concentrates

(entropy 0.19 and 4 % dead features at step 2000), but at step 2000 it holds at most 10 always-on features (Entry 6) and $f \leq 0.12$. This run and support scale 0.1 continue to 20k steps.

Correction to Entry 4. The hard reference of Entry 4 used `bottleneck_decoder_scale=backward_preserving` (the depth study’s base); the campaign base uses `none`. The exact hard backward on the campaign base (support scale 0) reaches 2.896 at step 500 against 2.912: the 0.02 advantage of support scale 0.1 in Entry 4 is mostly this difference. The support-scale-0 run is the reference from here on.

7 Entry 6 — what fills the support: always-on features

`code_usage.py` (on the box; 4 validation batches of 8×512 tokens, no gradient) records the code c after every gate: $n_{>1/2}$, the number of features active for more than half of the tokens; $E_{>1/2}$, their share of the code energy; f , the common-mode fraction of the code within a sequence (as in the depth study, positions $t \geq 16$); and the rank boundary b . Gates 6 / 12 / 18 / 24:

checkpoint	$n_{>1/2}$	$E_{>1/2}$	f	b
F1, $T = 1$, step 2000 (CE 5.9)	32 / 32 / 32 / 32	.95 / .93 / .91 / .95	.84 / .83 / .81 / .86	620 / 1015 / 3622
unmodified, $T = 1$, step 2000 (5.3)	27 / 25 / 26 / 24	.67 / .73 / .77 / .79	.53 / .67 / .71 / .80	175 / 520 / 149
support scale 0.1, step 2000	0 / 0 / 0 / 0	0 / 0 / 0 / 0	.01 / .01 / .01 / .05	1.7 / 1.9 / 2.1
F3 (inactive only), step 2000	0 / 1 / 2 / 10	0 / .02 / .10 / .29	.01 / .01 / .08 / .18	2.5 / 2.7 / 3.8
hard $K = 32$, step 16k	0 / 1 / 1 / 2	0 / .01 / .01 / .05	.02 / .02 / .02 / .06	1.8 / 2.1 / 2.5
hard $K = 512$, step 20k	30 / 59 / 88 / 83	.04 / .08 / .14 / .17	.04 / .04 / .06 / .07	1.6 / 2.0 / 2.3

Measured. In the collapsed F1 run every one of the $K = 32$ slots from gate 6 on is held by a feature that is active for more than half of the tokens; they carry 91–95 % of the code’s energy, the codes of all positions nearly coincide ($f = 0.81$ – 0.86), and their values reach 10^2 – 10^4 , growing with depth. The unmodified run is the same with 24–27 such features. The hard models keep at most 2 of 32 (and 17 % of the energy at $K = 512$); support scale 0.1 has none; F3 has 10 at the last gate holding 29 % of its energy — the same failure, slower.

Interpretation: a token collapse in code space. The code is the residual stream, and it stops depending on the token when the K slots are taken by features active for every token; the readout then sees only their token-independent values (CE $\approx$ the unigram entropy, 5.97). The surrogate creates them because it gives an *inactive* feature a gradient on its score, with a sign set by how much the loss would gain from including it. A generic feature — a direction useful for every token, which early in training is the unigram — receives a push of the same sign at every token, and the parameters that implement a token-independent score shift (the encoder row against the mean of the updates) raise it for all tokens at once, until it is active everywhere. In the code residual every block then adds its token-common update to an always-on feature, so its value grows with depth and with training, far above the kernel window, where neither the surrogate nor bounded optimizer steps bring it back. The hard gradient gives an inactive feature no score gradient at all: how often a feature is used follows only from value learning, and the hard models end with few always-on features holding little energy.

Fix to test (F4): center the support term over tokens. The boundary correction of `through_rank_kappa` already removes one common mode of the support term: the mean over one token’s pool, a shift of all of that token’s scores, which only moves its boundary. The collapse runs through the other one: the mean over the tokens of one feature’s term, a shift of that feature’s score for all tokens, which changes how often it is used. `rblapsum_center_tokens` (commit `ced6be9`) removes it: per gate and feature, the mean of the support term over the micro-batch’s tokens whose pool contains the feature is subtracted on those tokens. The surrogate then decides *which* tokens use a feature; *how often* it is used is left to the exact gradient, as in the hard model. Oracle:

IDENTICAL against 4beaf1b. **Prediction:** F1 and F3 at full strength with centering keep $n_{>1/2}$ near the hard model's and do not collapse.

Round 3 (2000 steps, $K = 32$, $J = 224$, $T = 1$):

run	change	tests
updinact_t1	F1 routing, inactive members only	which half of the term drives
updact_t1	F1 routing, active members only	
upd_t1_ctr	F1 + token centering	the fix at full strength
inact_t1_ctr	F3 + token centering	
onset runs	F1, support scale 0, F1 + centering; a checkpoint every 50 steps to 600	the timeline of $n_{>1/2}$, f , b

Measured: the onset (F1, $T = 1$, a checkpoint every 50 steps; $n_{>1/2}$ / $E_{>1/2}$ / f):

step (train CE)	gate 6	gate 12	gate 18	gate 24
50 (9.03)	0 / .00 / .05	0 / .00 / .06	5 / .15 / .16	18 / .65 / .62
100 (5.83)	0 / .00 / .04	0 / .00 / .04	3 / .07 / .08	24 / .70 / .65
200 (3.89)	25 / .48 / .46	8 / .06 / .08	9 / .10 / .09	18 / .31 / .24
300 (3.47)	31 / .87 / .85	15 / .11 / .12	21 / .27 / .26	14 / .22 / .20
450 (3.92)	32 / .96 / .94	32 / .83 / .81	32 / .92 / .91	12 / .23 / .24

The takeover starts at the *last* gate within 50 steps, while the model is learning the unigram (train CE 9), reaches gate 6 by step 200 and the middle gates by step 450; the last gate partly recovers.

The same series for the exact hard backward (support scale 0): the last gate forms always-on features in the unigram phase as well (10 features with 60 % of the energy at step 50, 18 with 69 % at step 100, $f = 0.64$), and they dissolve as training proceeds (6 with 10 % at step 600); gates 6, 12 and 18 never have any ($f \leq 0.05$). **Interpretation:** an always-on code at the readout during the unigram phase is the ordinary way the model represents the unigram. What the surrogate adds is the spread of always-on features to the lower gates and their lock-in.

Measured: round 3 (val CE at steps 500 / 1000 / 2000; the support-scale-0 run for reference: 2.896 / 2.435 / 2.104):

run	val CE	dead (step 1000)	note
F1 routing, active members only	3.141 / 3.025 / 4.918	0.73	collapse
F1 routing, inactive members only	2.920 / 2.525 / 2.280	0.01	behind by 0.18 at 2k
F1 + token centering	5.263 at 500	0.93 at step 200	stopped at step 500
F3 + token centering	5.778 at 500	0.90 at step 200	stopped at step 500

The ablation. Both halves of the routed term hurt; the active half is the destructive one (73 % of the features dead by step 1000, against 1 %; val CE 4.918 at step 2000), and the inactive half alone is 0.18 behind the hard backward at step 2000, with the boundary growing steadily (5.0 to 9.5). **Interpretation:** for an active feature the routed term multiplies the update's gradient by $1 + |u|\kappa \approx 3$ at the boundary while the carry keeps $m \odot g$; block ℓ then learns the values of its marginal features three times faster than the blocks that wrote them, see Entry 7.

The prediction for F4 failed, badly. Token centering does not prevent the collapse; it accelerates it: dead fraction 0.11 at step 100 and 0.93 at step 200, the boundary 4 to 12 by step 100, gradient norm ~ 10 from step 40 (the uncentered F1 reached these states only at step 300–500). The per-token zero-sum survives centering (the logged $|\sum_i g_{s,i}|/\|g_s\|$ stays $\leq 10^{-4}$), and at initialization centering changes σ by less than 1 % (the token mean of the support term is 0.4 % of its energy, the level of chance for ~ 256 tokens per feature). **Measured** (the centered F1 run with a

checkpoint every 50 steps, $n_{>1/2} / E_{>1/2} / f$ at gates 6 / 12 / 18 / 24): at step 50 $f = 0.12 / 0.13 / 0.22 / 0.44$ (uncentered F1: 0.05 / 0.06 / 0.16 / 0.62); at step 100 every gate holds 30–31 always-on features with 91–98% of the energy and $f = 0.90$ –0.96, at all gates at once, where the uncentered run started at the last gate and needed 450 steps to reach the middle ones. On checkpoints of the uncentered runs the support term’s per-feature token mean is at the level of chance (0.4–0.6% of its energy at gates 0–18, 2–5% at the last gate) and uncorrelated with how often a feature is used ($|r| \leq 0.09$).

Interpretation. Centering is not a small change even though the mean it removes is small: it moves that mean from the few tokens where the feature sits near the boundary to every token whose pool contains the feature, including those where it is active far above the boundary and the kernel is zero. There the term acts on the feature’s value, with the same sign at every token, so the gradient on each block’s output acquires a token-independent component, $W_E^\top$ (per-feature constant), at every gate at once; a token-independent update is exactly what feeds always-on features. The support term must stay where the kernel puts it. Token centering is abandoned.

8 Entry 7 — one support term per gradient path

Two routings have now been tested against the compounding of Entries 1–3: the unmodified one (each gate’s support term applied to the total upstream, which already contains the terms of later gates, so they multiply along the carry) and F1 (the term routed into the update only: no compounding, but a block gets no credit for how the features it writes are selected further up). F1 trains worse than the unmodified surrogate at every support scale tried (0.3: 2.189 against 2.064 at step 2000). The estimator the stack actually calls for sits between them. A hard TopK stack is piecewise linear; the surrogate estimates what a change of support would do. To first order in the support changes, one change at a time, the gradient is

$$\frac{\partial \mathcal{L}}{\partial \theta} \approx \underbrace{J_{\text{hard}}^\top g}_{\text{no support change}} + \sum_{\ell} \left(\frac{\partial u_{\ell}}{\partial \theta} \right)_{\text{hard}}^\top S_{\ell}^\top H_{\ell}, \quad (4)$$

where H_{ℓ} is the gradient that reaches gate ℓ ’s output along hard paths only, $S_{\ell}^\top$ the gate’s support term, and every other Jacobian is the hard one. The carry and the update receive gate ℓ ’s term alike (unlike F1), and it is never multiplied by another gate’s (unlike the unmodified backward, whose extra terms are products of support changes at several gates, second and higher order).

Stated as two requirements on any surrogate backward for this stack: **(R1) one gradient for the gate’s input** — c_{ℓ} and v_{ℓ} enter the gate only through $u = c_{\ell} + v_{\ell}$, so an estimator of $\partial \mathcal{L} / \partial u$ must give both the same gradient; F1 violates it (the carry gets $m \odot g$, the update $m \odot g + S_{\ell}^\top g$), so block ℓ and the blocks that wrote the carried code are trained on different estimates of the same derivative. **(R2) no products of support terms** — the hard forward has no such terms; the unmodified backward violates it, and the products grow along the carry. The unmodified backward at support scale s satisfies R1 and violates R2 only at order s^2 , which is the reading of why $s = 0.1$ –0.3 trains; F3 satisfies R1 and keeps products only along stretches where a feature stays just below the boundary ($|u| \kappa < 1$ there). The first-order estimator satisfies both at full strength.

`rblapsum_surrogate_scope=first_order` (commit 2965fa6) computes it with two backward passes per micro-batch: a hard pass (every gate passes back $m \odot g$ and keeps $g = H_{\ell}$), as `torch.autograd.grad` to the token embedding so that no parameter’s `.grad` is touched, then the ordinary pass, in which each gate adds the support term computed from its kept H_{ℓ} . Unit tests: against the explicit two-gate formula, and, on a 3-layer code-residual model (with its read paths), against the hard gradient plus the sum over the gates of (gradient with only that gate relaxed – hard gradient), equal to float64 round-off; oracle IDENTICAL against 4beaf1b (hard

and rbk). The extra pass costs about 45 % of a step (1.57 against 1.09 s at $K = 32$). At initialization the per-block factor of $\partial\mathcal{L}/\partial c$ is $\times 0.982$ (hard 1.072, unmodified 0.643): what remains is the additive sum of the later gates' terms, not a product.

Round 4: first order at support scale 1 and 0.3 ($K = 32$, $J = 224$, $T = 1$), 2000 steps.

run	val CE 500 / 1000 / 1500 / 2000	– hard at 2k	dead (2k)	entropy (2k)	b (2k)
hard backward (support scale 0)	2.896 / 2.435 / 2.232 / 2.104	0	0.000	0.56	2.7
first order, scale 1	2.912 / 2.417 / 2.174 / 2.028	−0.076	0.000	0.36	2.1
first order, scale 0.3	2.877 / 2.410 / 2.195 / 2.054	−0.050	0.000	0.48	2.1
unmodified, scale 0.3	2.937 / 2.440 / 2.206 / 2.064	−0.040	0.04	0.19	2.7
unmodified, scale 0.1	2.890 / 2.428 / 2.216 / 2.089	−0.014	0.000	0.62	2.2

At step 2000 the first-order run has at most 2 always-on features per gate ($E_{>1/2} \leq 0.05$, $f \leq 0.15$ at the last gate, ≤ 0.05 below).

Measured. The first-order estimator trains at full strength, with no dead features, the boundary falling like the hard model's, and is the best run at steps 1000, 1500 and 2000: 0.076 below the exact hard backward at step 2000, against 0.040 for the best rescaling of the unmodified backward. At the same scale 0.3 the first-order estimator is ahead of the unmodified one by 0.010. Its feature usage concentrates (entropy 0.36 against 0.56) without dead features or always-on features.

Interpretation. The estimator the code-residual stack needs is the sum of single-gate relaxations, not their product: the product's extra terms (R2) were what made the unmodified surrogate fail, and the routing's two gradients for one input (R1) what made the routed one fail. The first-order estimator is the hard gradient plus the sum over the gates of (the backward pass in which only gate ℓ uses its surrogate Jacobian – the hard one); every one of these is a backward pass through the gate's input u_ℓ , so R1 holds by construction. Damping the product (support scale) approximates it to first order in the scale and pays for the second-order terms with a smaller first-order term.

Continuations: the first-order run (scale 1) to 20k steps; the hard backward and the unmodified backward at scales 0.1 and 0.3 are already continuing. Val CE minus the hard backward's, at steps 1k / 2k / 3k / 4k / 5k / 6k / 7k / 8k / 9k:

	1k	2k	3k	4k	5k	6k	7k	8k	9k
first order, scale 1	−0.018	−0.076	−0.074	−0.068	−0.065				
unmodified, scale 0.3	+0.006	−0.040	−0.048	−0.049	−0.046	−0.036	−0.033	−0.028	
unmodified, scale 0.1	−0.006	−0.014	−0.018	−0.015	−0.015	−0.013	−0.011	−0.009	−0.008

Measured: every surrogate's advantage peaks between 2k and 4k steps and then shrinks; the hard backward is catching up.

The 20k values (val CE, difference to the hard backward): hard 1.5020; first order 1.4569 (−0.045; −0.050 at 10k, −0.048 at 16k); first order at $T = 2$ 1.4657 (−0.036; −0.088 at 2k); unmodified at scale 0.3 1.4856 (−0.016); at scale 0.1 1.4961 (−0.006). The first-order advantage shrinks during the first 10k steps and is nearly constant after; the damped unmodified backward keeps little. A second seed (initialization and data order) at step 2000: hard 2.0821, first order 2.0254 (−0.057), first order at $T = 2$ 2.0125 (−0.070); first seed −0.076 and −0.088. Generality screens: first order at $K = 256$, $J = 256$ (with its hard control), at $J = 480$, and at $T = 2$; first order with per-block projections.

9 Entry 8 — does the first-order estimator carry over?

2000-step screens of the first-order estimator (support scale 1) away from $K = 32$, $J = 224$, $T = 1$, each against the exact hard backward of the same configuration (J and T do not enter the hard backward, so the $K = 32$ control serves all $K = 32$ rows):

configuration	first order, val CE 500 / 1000 / 1500 / 2000	hard backward at 2000
$K = 32$, $J = 224$, $T = 1$ (Entry 7)	2.912 / 2.417 / 2.174 / 2.028	2.104
$K = 32$, $J = 224$, $T = 1$, per-block projections	2.869 / 2.369 / 2.134 / 1.992	—
$K = 256$, $J = 256$, $T = 1$	2.869 / 2.350 / 2.098 / 1.958	1.870
$K = 32$, $J = 480$, $T = 1$	2.919 / 2.430 / 2.184 / 2.040	2.104
$K = 32$, $J = 224$, $T = 2$	2.877 / 2.391 / 2.158 / 2.016	2.104
$K = 256$, $J = 256$, $T = 1$, inactive members only	2.798 / 2.257 / 2.015 / 1.891	1.870
$K = 256$, $J = 256$, $T = 1$, support scale 0.3	2.816 / 2.284 / 2.048 / 1.925	1.870
$K = 256$, $J = 256$, $T = 0.3b$ (scale-free)	2.850 / 2.329 / 2.085 / 1.949	1.870

Measured: $K = 256$. At $K = 256$, $J = 256$ the first-order estimator at full strength is *worse* than the hard backward at every evaluation (+0.111, +0.129, +0.109, +0.088 at steps 500–2000). Its code holds many features that are active for more than half of the tokens, 164 / 153 / 67 of the 256 slots at gates 6 / 12 / 18 (hard: 5 / 10 / 15), but they carry little energy (10–15 %) and the codes of different positions stay distinct ($f \leq 0.15$); the rank boundary is lower (1.6 against 2.7–3.2) and the feature-usage entropy falls (0.51 against 0.77). At initialization its surrogate energy is lower than at $K = 32$ ($\sigma = 0.080$ against 0.18) and the gradient reaching the code grows $\times 1.08$ per block relative to hard, as at $K = 32$. On the step-2000 checkpoints the per-feature token mean of the support term is 0.5–1.4 % of its energy and uncorrelated with usage ($|r| \leq 0.02$), close to chance.

The code’s scale at the boundary, step-2000 checkpoints, `surrogate_probe.py` (mean over gates; window = pool members with $|s - b| < T = 1$):

checkpoint	boundary b	RMS of the kept scores	pool members in the window
hard backward, $K = 32$	2.71	10.8	173 of 256
first order, $K = 32$	2.08	5.1	227 of 256
hard backward, $K = 256$	3.26	11.3	23 of 512
first order, $K = 256$	2.08	4.9	428 of 512

With the first-order estimator the code is half as large and its scores crowd around a lower boundary, at both K ; with $K = 256$ the hard model keeps its boundary region nearly empty (23 of 512 within $\pm T$), the first-order model fills it (428).

Interpretation (to be tested). The always-on features at $K = 256$ are not the token collapse of Entry 6: they are weak and many, sitting just above a boundary that has moved down. This is what a compressed boundary band looks like (`docs/rblapsum-kappa-stability-neutral.tex`): the support term pushes marginal active features down and near candidates up, toward the boundary, and with $K + J = 512$ of 4096 features in the pool many features end up just above it for most tokens, occupying slots with little content. With a fixed T , a crowded, smaller code puts more of the pool inside the kernel, which strengthens the push: a loop. Tests: the same at support scale 0.3; the first-order estimator restricted to the inactive members (no push on active features); and the first-order estimator with the scale-free kernel ($T = 0.3b$, the $T = 1$ kernel at initialization).

Measured: the other screens. At $J = 480$ the first-order estimator gains 0.064 at step 2000 (0.076 at $J = 224$), with at most 2 always-on features per gate. With an encoder/decoder pair per block it reaches 1.992 at step 2000 (its hard control was dropped for GPU time; the shared-pair comparison is the one above). At $K = 256$ a smaller support scale does not rescue it: at scale

0.3 the first-order estimator is +0.059, +0.063, +0.059 behind the hard backward at steps 500, 1000, 1500, the unmodified backward +0.118, +0.128, +0.112, +0.098 at steps 500–2000. The first-order estimator restricted to the inactive members is the closest, +0.041 at step 500, with the usage entropy of the hard model (0.85 at step 400). **Measured: how crowded the hard model’s boundary is.** Pool members within $\pm T$ of the boundary, hard backward: at $K = 32$ 173 / 193 / 208 of 256 at steps 2k / 4k / 6k (the region fills further during training); at $K = 256$ 23 of 512 at step 2k (it empties, from 310 at initialization). The first-order model at $K = 32$: 227 and 235 of 256 at steps 2k and 4k.

Interpretation. The two K differ in what the hard model does with its boundary. At $K = 32$ of 4096 the boundary sits in the tail of the score distribution, the competition for the last slots stays close, and information about the candidates just below it (what the support term supplies) is what the hard gradient lacks. At $K = 256$ the hard model separates the selected features from the rest: their values grow under the hard gradient while the others get none, and the boundary region empties. The support term works against that separation — its corrections are exchanges that pull both sides toward the boundary — and a crowded boundary in a hard forward means unstable selections and slots held by weak features (Entry 8 above). The surrogate helps where the selection is competitive and hurts where it can separate; at a given N which regime holds depends on K .

Measured: the active-member term at $K = 256$. Restricted to the inactive members, the first-order estimator at $K = 256$ is +0.041, +0.036, +0.026, +0.020 behind the hard backward at steps 500–2000, closing; its code looks like the hard model’s: 5 / 7 / 13 always-on features at gates 6 / 12 / 18 (hard 5 / 10 / 15, full pool 164 / 153 / 67) and a boundary at 3.0–3.5 (hard 2.7–3.2, full pool 1.6). At $K = 32$ the same restriction loses the whole gain: +0.016, +0.021, +0.007, +0.001 at steps 500–2000 (val CE 2.105 at step 2000), against -0.076 for the full pool. The scale-free kernel does not help at $K = 256$ (+0.092, +0.109, +0.096, +0.078 at steps 500–2000). At $T = 2$ the first-order estimator gains more at $K = 32$: -0.088 at step 2000 ($T = 1$: -0.076).

Interpretation. The term on active features — the one that pushes marginal selected features toward the boundary — is what crowds the boundary at $K = 256$: without it the code keeps the hard model’s separation and the deficit closes; with it the surrogate is best at $K = 32$, where the selection never separates. The candidates’ term alone is the safe part of the surrogate; the active features’ term pays only where the competition for the last slots stays close.

The $K = 256$ runs to 20k (val CE, difference to the hard backward 1.3067): first order 1.3844 (+0.078; +0.069 at 10k, +0.075 at 16k); first order on the inactive members 1.2998 (-0.007 ; +0.004 at 10k, -0.000 at 12k, -0.006 at 16k). **Measured:** the full first-order estimator does not close its gap at $K = 256$; the inactive-member version crosses near step 12k and ends slightly ahead. At $K = 32$ the inactive-member version had no gain at 2k; the variant that helps depends on K .

10 Summary

1. **Why the unmodified surrogate fails.** The code residual carries each code coordinate through an identity, and the unmodified backward applies every gate’s support term to the total upstream, which already contains the terms of the gates above: along a feature carried near the boundary they multiply, $\prod(1 + |u|\kappa) \approx 3^n$ (Entries 1–2). These products are terms of second and higher order in the changes of support, which a hard forward does not have. The gradient reaching the entry gate is $2.6 \cdot 10^5$ times the hard model’s at initialization; in training the code collapses onto always-on features and dead ones (Entry 6).
2. **What the existing knobs do.** The support scale reduces the products to order γ^2 and trains for $\gamma \leq 0.3$ ($\gamma = 0.3$: -0.040 at 2k, shrinking to -0.028 at 8k; Entry 5). The temperature, the

window and the boundary mode leave the products in place (Entry 1); a scale-free kernel does not stop the routed collapse (Entry 5).

3. **A tempting fix that fails.** Routing the support term into the update only removes the products but gives the carry and the update different gradients for one input; the code fills with always-on features (Entries 4, 6). Centering the support term over tokens breaks the kernel’s locality and fails faster (Entry 6).
4. **The principled fix.** The first-order estimator (`rblapsum_surrogate_scope=first_order`): the hard gradient plus, for each gate, the change its support term makes when every other gate is exact; one backward pass through each gate’s input (R1) and no products (R2) (Entry 7). Best at $K = 32$ (-0.076 at 2k; -0.064 at $J = 480$); the advantage shrinks with training (-0.065 at 5k).
5. **To 20k steps.** At $K = 32$ the first-order estimator ends 0.045 below the hard backward (1.457 against 1.502), the damped unmodified backward 0.016 below (scale 0.3). At $K = 256$ the full first-order estimator stays 0.078 behind; restricted to the inactive members it ends 0.007 ahead. The term on active features helps where the hard model’s selection stays competitive at the boundary ($K = 32$) and hurts where the hard model separates its selected features ($K = 256$) (Entry 8).